

```
In [ ]: import pandas as pd
```

```
In [3]: df = pd.read_csv('fraud_data.csv')
```

```
In [5]: df.head()
```

```
Out[5]:
```

	trans_date_trans_time	merchant	category	amt	city	state	lat	lon
0	04-01-2019 00:58	"Stokes, Christiansen and Sipes"	grocery_net	14.37	Wales	AK	64.7556	-165.6072
1	04-01-2019 15:06	Predovic Inc	shopping_net	966.11	Wales	AK	64.7556	-165.6072
2	04-01-2019 22:37	Wisozk and Sons	misc_pos	49.61	Wales	AK	64.7556	-165.6072
3	04-01-2019 23:06	Murray-Smitham	grocery_pos	295.26	Wales	AK	64.7556	-165.6072
4	04-01-2019 23:59	Friesen Lt	health_fitness	18.17	Wales	AK	64.7556	-165.6072

```
In [7]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 14446 entries, 0 to 14445
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   trans_date_trans_time  14446 non-null  object 
1   merchant               14446 non-null  object 
2   category               14446 non-null  object 
3   amt                   14446 non-null  float64
4   city                  14446 non-null  object 
5   state                 14446 non-null  object 
6   lat                   14446 non-null  float64
7   long                  14446 non-null  float64
8   city_pop              14446 non-null  int64  
9   job                   14446 non-null  object 
10  dob                   14446 non-null  object 
11  trans_num              14446 non-null  object 
12  merch_lat              14446 non-null  float64
13  merch_long             14446 non-null  float64
14  is_fraud               14446 non-null  object 
dtypes: float64(5), int64(1), object(9)
memory usage: 1.7+ MB
```

```
In [9]: df.describe()
```

Out[9]:

	amt	lat	long	city_pop	merch_lat	merch_l
count	14446.000000	14446.000000	14446.000000	1.444600e+04	14446.000000	14446.000
mean	124.430073	39.787692	-110.874225	1.065370e+05	39.787991	-110.874
std	231.352587	5.317039	12.985813	2.902916e+05	5.360593	12.995
min	1.000000	20.027100	-165.672300	4.600000e+01	19.032689	-166.670
25%	12.080000	36.715400	-120.415800	4.930000e+02	36.794655	-120.146
50%	51.520000	39.666200	-111.098500	1.645000e+03	39.620953	-111.192
75%	101.030000	41.940400	-101.136000	3.543900e+04	42.275740	-100.446
max	3261.470000	66.693300	-89.628700	2.383912e+06	67.510267	-88.646

```
In [11]: pd.set_option('display.max_columns', None)
df.head()
```

Out[11]:

	trans_date_trans_time	merchant	category	amt	city	state	lat	lon
0	04-01-2019 00:58	"Stokes, Christiansen and Sipes"	grocery_net	14.37	Wales	AK	64.7556	-165.6
1	04-01-2019 15:06	Predovic Inc	shopping_net	966.11	Wales	AK	64.7556	-165.6
2	04-01-2019 22:37	Wisozk and Sons	misc_pos	49.61	Wales	AK	64.7556	-165.6
3	04-01-2019 23:06	Murray-Smitham	grocery_pos	295.26	Wales	AK	64.7556	-165.6
4	04-01-2019 23:59	Friesen Lt	health_fitness	18.17	Wales	AK	64.7556	-165.6

```
In [13]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
In [14]: sns.set(style="whitegrid")
%matplotlib inline
```

```
In [17]: print(df.head())
print(df.info())
print(df.describe())
```

	trans_date_trans_time	merchant	category \
0	04-01-2019 00:58	"Stokes, Christiansen and Sipes"	grocery_net
1	04-01-2019 15:06	Predovic Inc	shopping_net
2	04-01-2019 22:37	Wisozk and Sons	misc_pos
3	04-01-2019 23:06	Murray-Smitham	grocery_pos
4	04-01-2019 23:59	Friesen Lt	health_fitness

	amt	city	state	lat	long	city_pop \
0	14.37	Wales	AK	64.7556	-165.6723	145
1	966.11	Wales	AK	64.7556	-165.6723	145
2	49.61	Wales	AK	64.7556	-165.6723	145
3	295.26	Wales	AK	64.7556	-165.6723	145
4	18.17	Wales	AK	64.7556	-165.6723	145

	job	dob	trans_num \
0	"Administrator, education"	09-11-1939	a3806e984cec6ac0096d8184c64ad3a1
1	"Administrator, education"	09-11-1939	a59185fe1b9ccf21323f581d7477573f
2	"Administrator, education"	09-11-1939	86ba3a888b42cd3925881fa34177b4e0
3	"Administrator, education"	09-11-1939	3a068fe1d856f0ecedbed33e4b5f4496
4	"Administrator, education"	09-11-1939	891cdd1191028759dc20dc224347a0ff

	merch_lat	merch_long	is_fraud
0	65.654142	-164.722603	1
1	65.468863	-165.473127	1
2	65.347667	-165.914542	1
3	64.445035	-166.080207	1
4	65.447094	-165.446843	1

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 14446 entries, 0 to 14445

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	trans_date_trans_time	14446 non-null	object
1	merchant	14446 non-null	object
2	category	14446 non-null	object
3	amt	14446 non-null	float64
4	city	14446 non-null	object
5	state	14446 non-null	object
6	lat	14446 non-null	float64
7	long	14446 non-null	float64
8	city_pop	14446 non-null	int64
9	job	14446 non-null	object
10	dob	14446 non-null	object
11	trans_num	14446 non-null	object
12	merch_lat	14446 non-null	float64
13	merch_long	14446 non-null	float64
14	is_fraud	14446 non-null	object

dtypes: float64(5), int64(1), object(9)

memory usage: 1.7+ MB

None

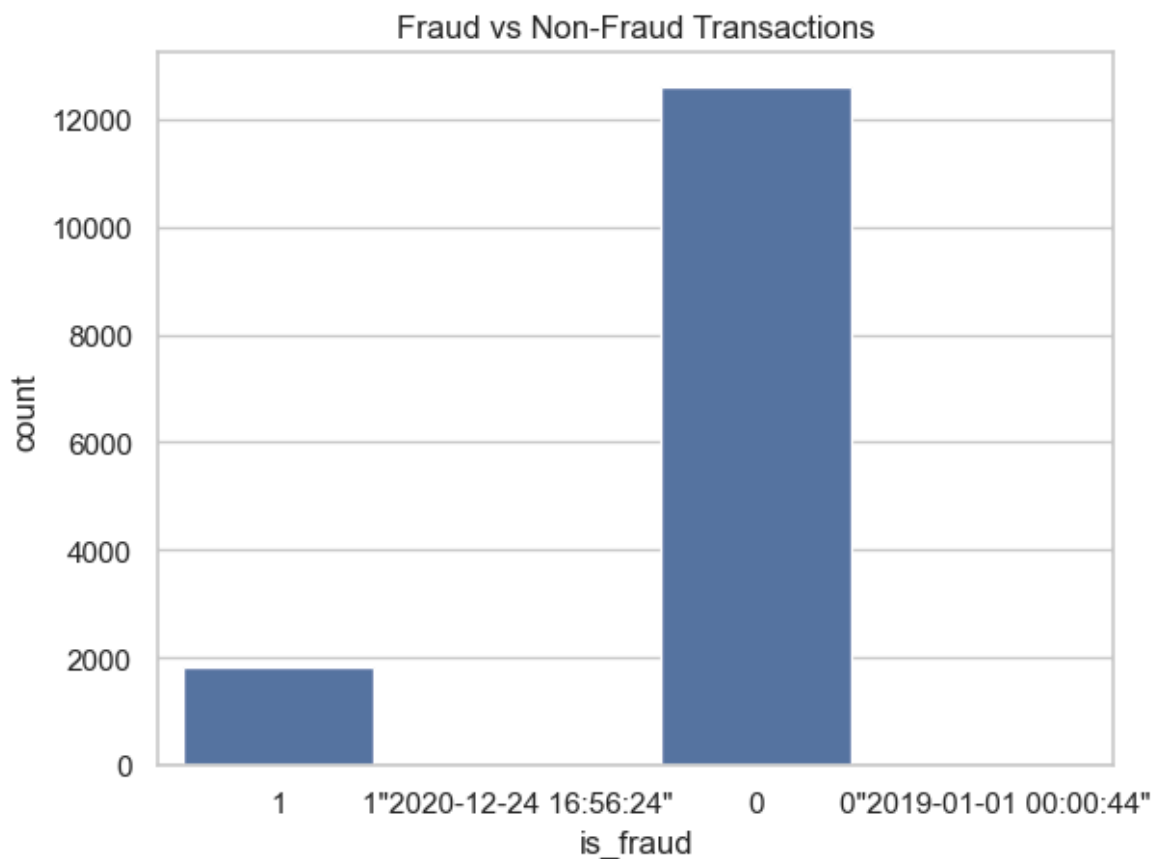
	amt	lat	long	city_pop	merch_lat \
count	14446.000000	14446.000000	14446.000000	1.444600e+04	14446.000000
mean	124.430073	39.787692	-110.874225	1.065370e+05	39.787991
std	231.352587	5.317039	12.985813	2.902916e+05	5.360593
min	1.000000	20.027100	-165.672300	4.600000e+01	19.032689
25%	12.080000	36.715400	-120.415800	4.930000e+02	36.794655
50%	51.520000	39.666200	-111.098500	1.645000e+03	39.620953
75%	101.030000	41.940400	-101.136000	3.543900e+04	42.275740
max	3261.470000	66.693300	-89.628700	2.383912e+06	67.510267

```
merch_long
count    14446.000000
mean     -110.874892
std        12.995596
min      -166.670685
25%      -120.146253
50%      -111.192629
75%      -100.446822
max       -88.646366
```

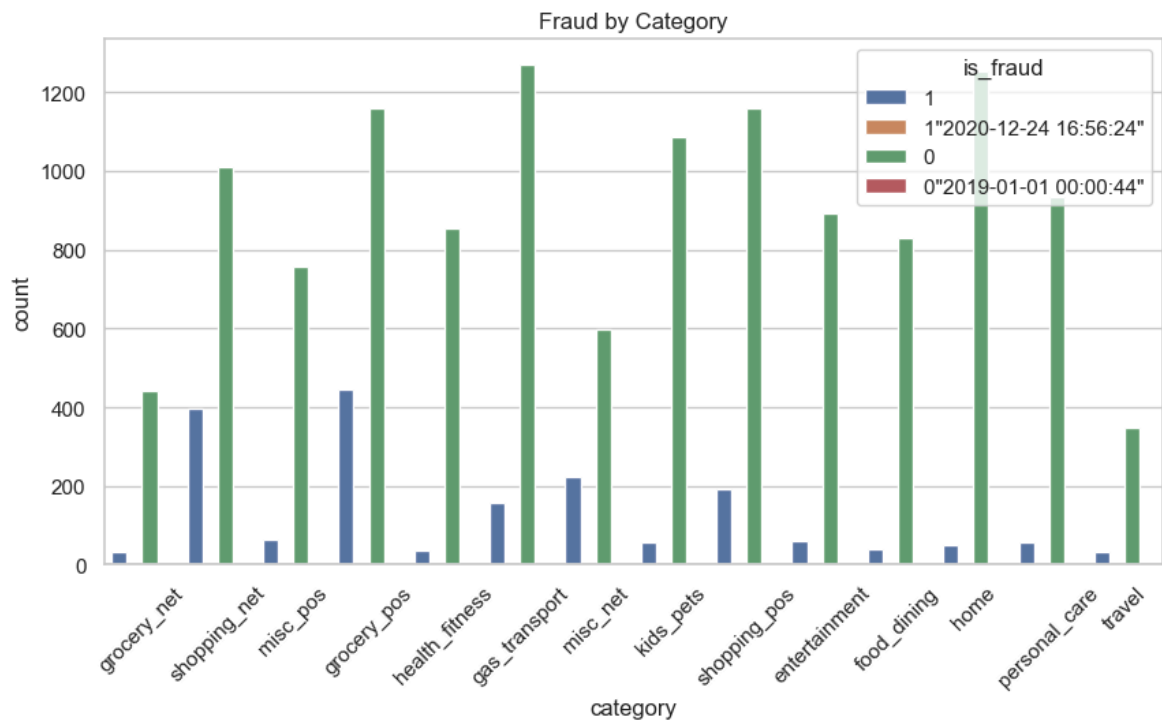
```
In [19]: print(df.isnull().sum())
```

```
trans_date_trans_time    0
merchant                 0
category                 0
amt                     0
city                    0
state                   0
lat                     0
long                   0
city_pop                0
job                     0
dob                     0
trans_num                0
merch_lat                0
merch_long               0
is_fraud                 0
dtype: int64
```

```
In [25]: sns.countplot(x='is_fraud', data=df)
plt.title('Fraud vs Non-Fraud Transactions')
plt.show()
```



```
In [29]: plt.figure(figsize=(10,5))
sns.countplot(x='category', hue='is_fraud', data=df)
plt.title('Fraud by Category')
plt.xticks(rotation=45)
plt.show()
```



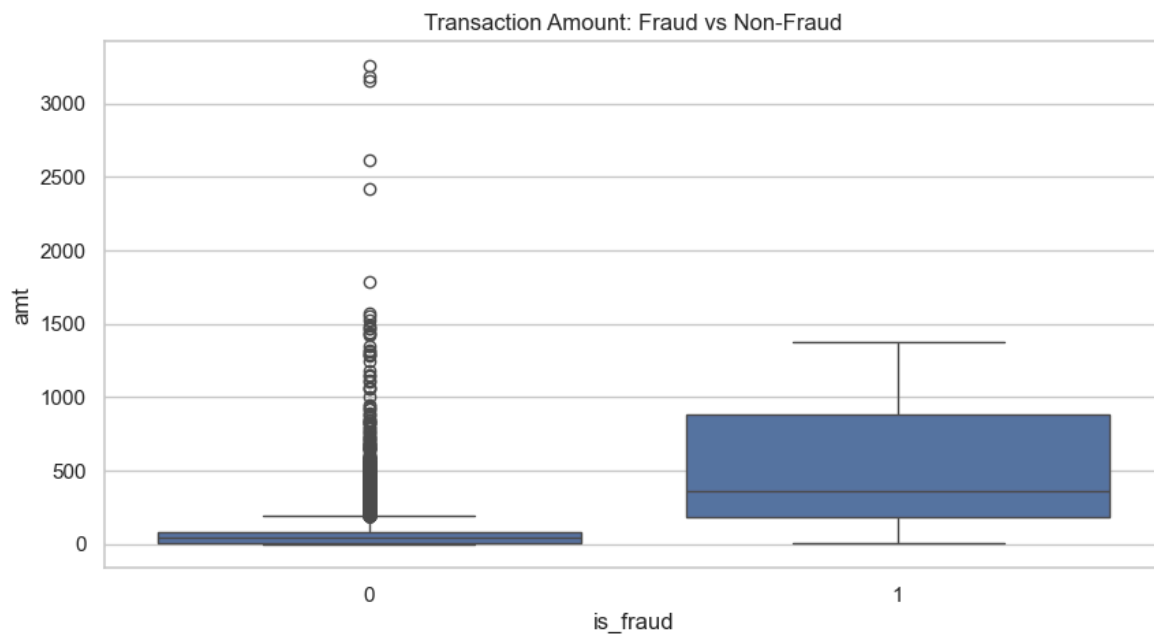
```
In [33]: df['is_fraud'].unique()
```

```
Out[33]: array(['1', '1"2020-12-24 16:56:24"', '0', '0"2019-01-01 00:00:44"'],
      dtype=object)
```

```
In [35]: df['is_fraud'] = pd.to_numeric(df['is_fraud'], errors='coerce')
```

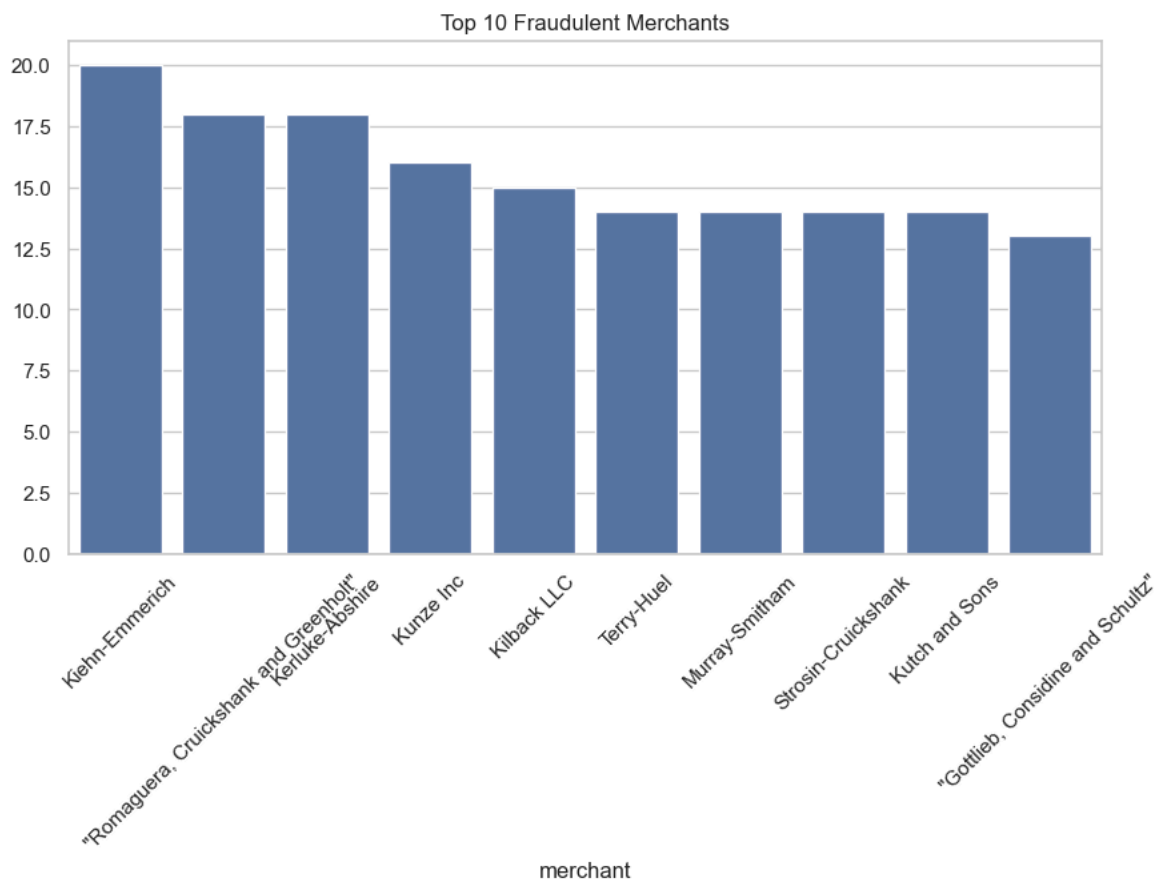
```
In [37]: df = df.dropna(subset=['is_fraud'])
df['is_fraud'] = df['is_fraud'].astype(int)
```

```
In [39]: plt.figure(figsize=(10,5))
sns.boxplot(x='is_fraud', y='amt', data=df)
plt.title('Transaction Amount: Fraud vs Non-Fraud')
plt.show()
```



```
In [43]: top_fraud_merchants = df[df['is_fraud']==1]['merchant'].value_counts().head(10)
plt.figure(figsize=(10,5))
sns.barplot(x=top_fraud_merchants.index, y=top_fraud_merchants.values)
plt.title('Top 10 Fraudulent Merchants')
plt.xticks(rotation=45)
plt.show()
```

<Figure size 1000x500 with 0 Axes>

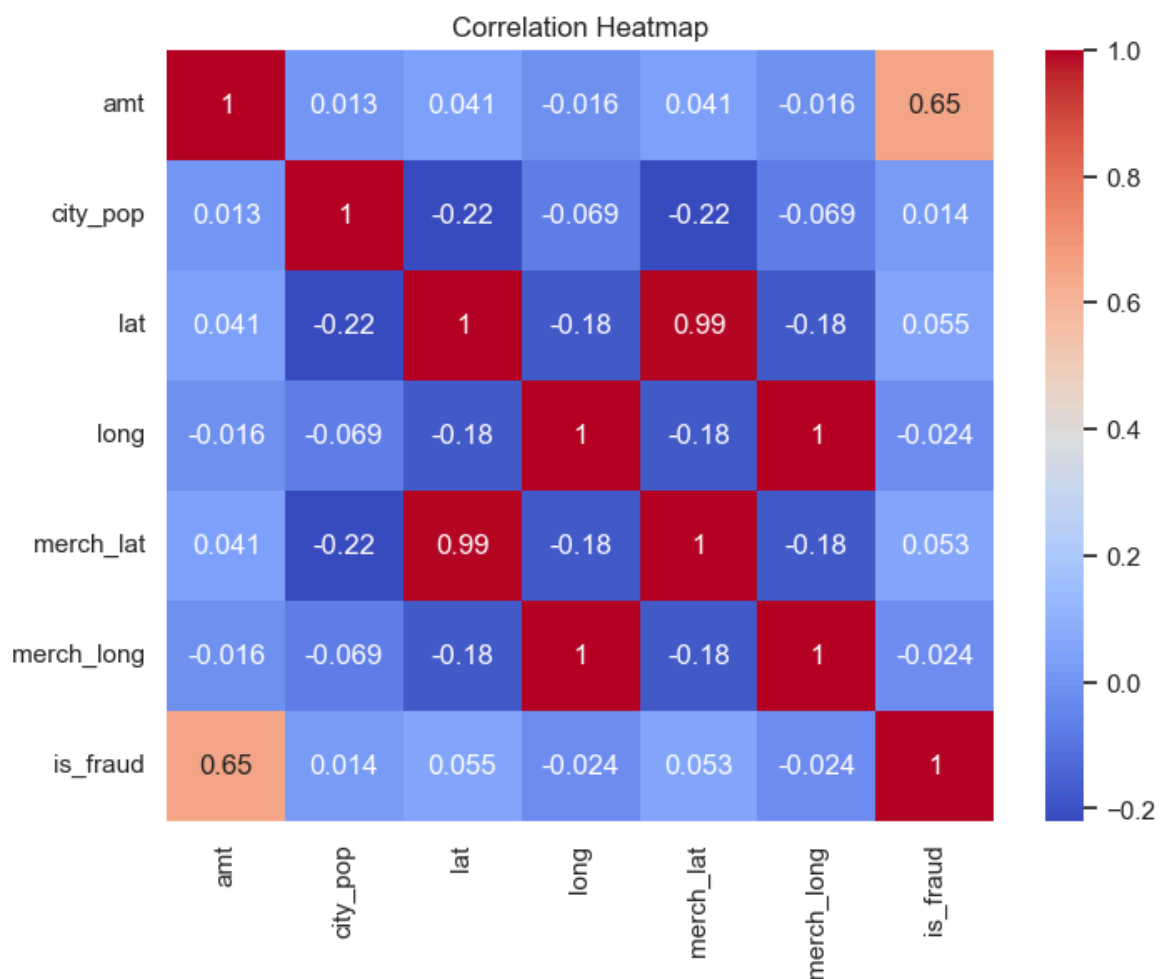


```
In [45]: plt.figure(figsize=(12,5))
sns.countplot(x='state', hue='is_fraud', data=df)
plt.title('Fraud by State')
```

```
plt.xticks(rotation=45)
plt.show()
```



```
In [47]: numeric_cols = ['amt', 'city_pop', 'lat', 'long', 'merch_lat', 'merch_long']
plt.figure(figsize=(8,6))
sns.heatmap(df[numeric_cols + ['is_fraud']].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```



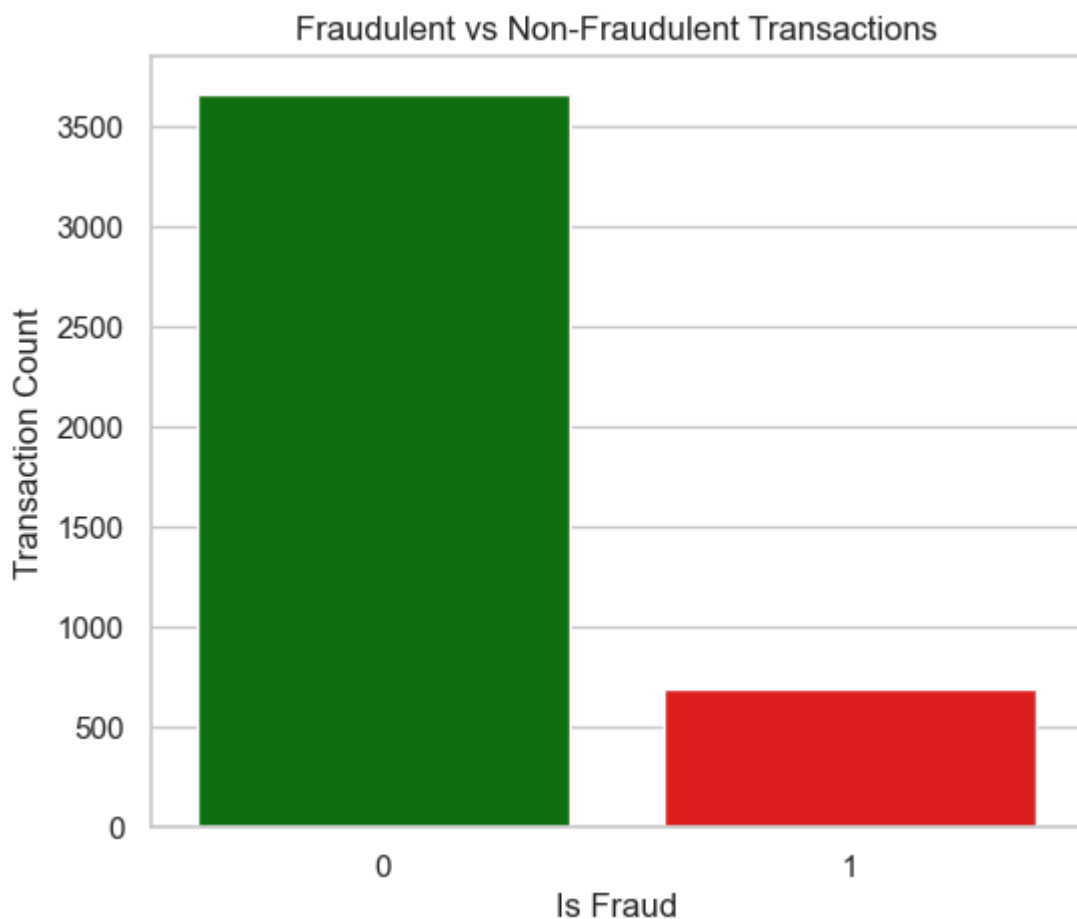
```
In [57]: df['lat'] = pd.to_numeric(df['lat'], errors='coerce')
df['long'] = pd.to_numeric(df['long'], errors='coerce')
```

In []:

```
In [61]: df['trans_date_trans_time'] = pd.to_datetime(df['trans_date_trans_time'], errors
df['amt'] = pd.to_numeric(df['amt'], errors='coerce')
df['city_pop'] = pd.to_numeric(df['city_pop'], errors='coerce')
df['lat'] = pd.to_numeric(df['lat'], errors='coerce')
df['long'] = pd.to_numeric(df['long'], errors='coerce')
```

```
In [63]: df = df.dropna(subset=['trans_date_trans_time', 'amt', 'lat', 'long', 'is_fraud'])
```

```
In [67]: plt.figure(figsize=(6,5))
sns.countplot(x='is_fraud', data=df, hue='is_fraud', palette={0:'green', 1:'red'})
plt.title("Fraudulent vs Non-Fraudulent Transactions")
plt.xlabel("Is Fraud")
plt.ylabel("Transaction Count")
plt.legend([],[], frameon=False) # remove duplicate legend
plt.show()
```



```
In [77]: df = pd.read_csv("fraud_data.csv")
df.columns = df.columns.str.strip()
```

```
In [79]: fraud_locations = df[df['is_fraud'] == 1].copy()
```

```
In [81]: fraud_locations['lat'] = pd.to_numeric(fraud_locations['lat'], errors='coerce')
fraud_locations['long'] = pd.to_numeric(fraud_locations['long'], errors='coerce')
fraud_locations = fraud_locations.dropna(subset=['lat', 'long'])
```

```
In [86]: import pandas as pd
import plotly.express as px
```

```
import plotly.io as pio
```

```
In [88]: pio.renderers.default = "notebook"
```

```
In [90]: df = pd.read_csv("fraud_data.csv")
df.columns = df.columns.str.strip()
```

```
In [92]: fraud_locations = df[df['is_fraud'] == 1].copy()
```

```
In [94]: fraud_locations['lat'] = pd.to_numeric(fraud_locations['lat'], errors='coerce')
fraud_locations['long'] = pd.to_numeric(fraud_locations['long'], errors='coerce')
fraud_locations = fraud_locations.dropna(subset=['lat', 'long'])
```

```
In [96]: fig = px.density_mapbox(
    fraud_locations,
    lat='lat',
    lon='long',
    radius=10,
    center=dict(lat=fraud_locations['lat'].mean(), lon=fraud_locations['long'].mean()),
    zoom=4,
    mapbox_style="stamen-terrain",
    title="Fraudulent Transactions Heatmap"
)
fig.show()
```

```
In [118]: # Ensure amt and is_fraud are numeric
df['amt'] = pd.to_numeric(df['amt'], errors='coerce') # invalid values become NaN
```

```
df['is_fraud'] = pd.to_numeric(df['is_fraud'], errors='coerce')

# Drop rows where these columns are NaN
df = df.dropna(subset=['amt', 'is_fraud'])
```

In [120...

```
import pandas as pd

bins = [0, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]
labels = ['0-50', '50-100', '100-500', '500-1k', '1k-5k', '5k-10k', '10k-50k', '50k+']
df['amt_bin'] = pd.cut(df['amt'], bins=bins, labels=labels, include_lowest=True)
```

C:\Users\karan\AppData\Local\Temp\ipykernel_12252\2504589704.py:5: SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

In [122...

```
df = df[df['is_fraud'].notna()] # slice
df['amt_bin'] = pd.cut(df['amt'], bins=bins, labels=labels) # warning occurs here
```

In [124...

```
# Filter numeric and drop invalid rows
df_clean = df.copy()
df_clean['amt'] = pd.to_numeric(df_clean['amt'], errors='coerce')
df_clean['is_fraud'] = pd.to_numeric(df_clean['is_fraud'], errors='coerce')
df_clean = df_clean.dropna(subset=['amt', 'is_fraud']).copy() # create a true copy

# Create bins safely
bins = [0, 50, 100, 500, 1000, 5000, 10000, 50000, 100000]
labels = ['0-50', '50-100', '100-500', '500-1k', '1k-5k', '5k-10k', '10k-50k', '50k+']
df_clean['amt_bin'] = pd.cut(df_clean['amt'], bins=bins, labels=labels, include_lowest=True)
```

In [128...

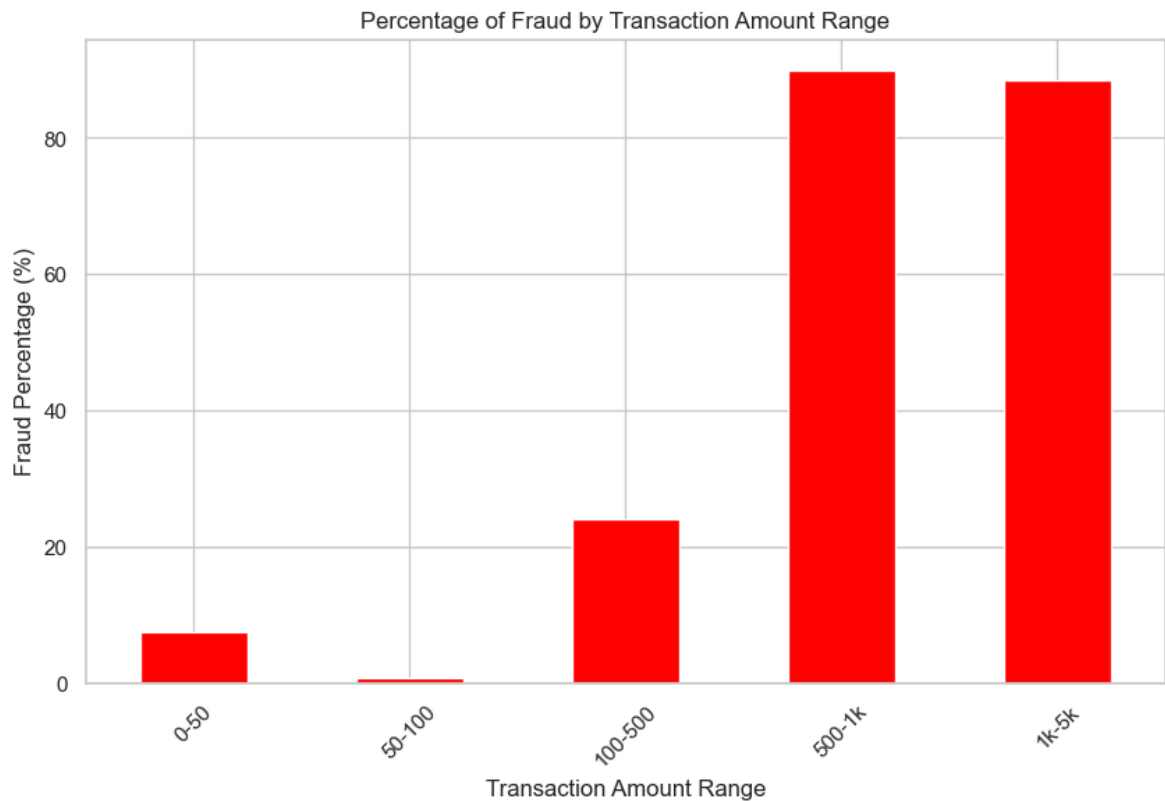
```
fraud_pct = df_clean.groupby('amt_bin', observed=True)['is_fraud'].mean() * 100
```

In [130...

```
import matplotlib.pyplot as plt

fraud_pct = df_clean.groupby('amt_bin', observed=True)['is_fraud'].mean() * 100

plt.figure(figsize=(10,6))
fraud_pct.plot(kind='bar', color='red')
plt.title("Percentage of Fraud by Transaction Amount Range")
plt.xlabel("Transaction Amount Range")
plt.ylabel("Fraud Percentage (%)")
plt.xticks(rotation=45)
plt.show()
```



In [150...

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Ensure numeric amt and is_fraud
df['amt'] = pd.to_numeric(df['amt'], errors='coerce')
df['is_fraud'] = pd.to_numeric(df['is_fraud'], errors='coerce')
df_clean = df.dropna(subset=['amt', 'is_fraud']).copy()

# Aggregate: count of frauds and avg amount per category
category_stats = df_clean.groupby('category', observed=True).agg(
    fraud_count=('is_fraud', 'sum'),
    avg_amt=('amt', 'mean')
).sort_values('fraud_count', ascending=False).head(10)

# Plot
fig, ax1 = plt.subplots(figsize=(12,6))

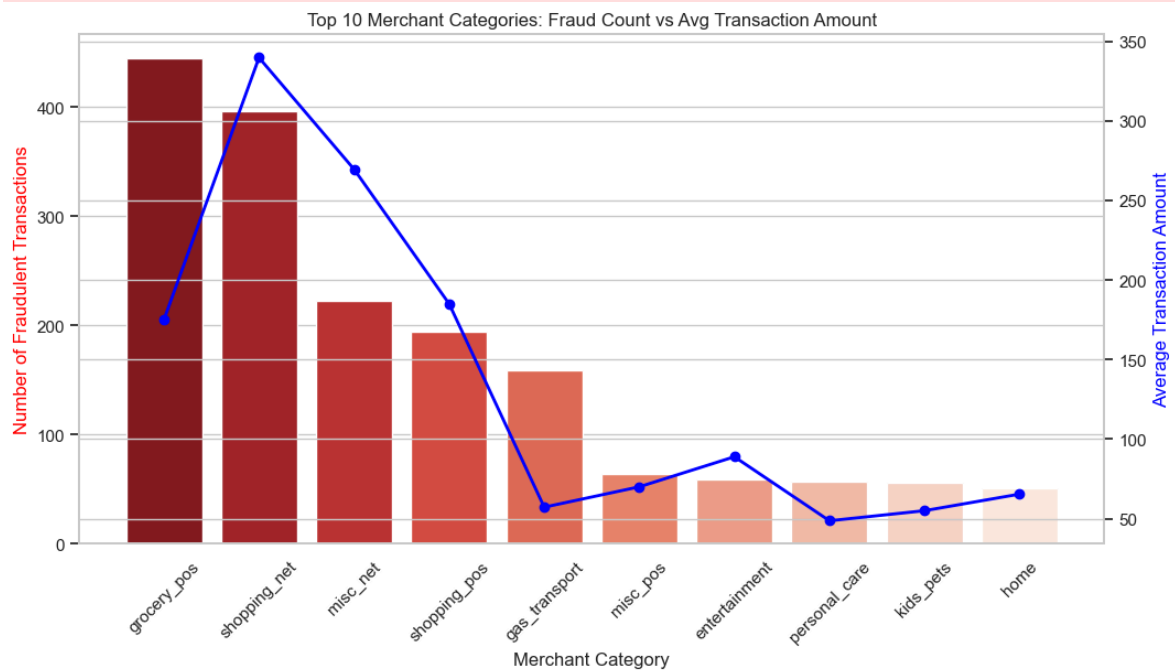
# Bar chart: fraud count
sns.barplot(x=category_stats.index, y=category_stats['fraud_count'], palette='Red')
ax1.set_ylabel("Number of Fraudulent Transactions", color='red')
ax1.set_xlabel("Merchant Category")
ax1.tick_params(axis='x', rotation=45)

# Line chart: average amount
ax2 = ax1.twinx()
ax2.plot(category_stats.index, category_stats['avg_amt'], color='blue', marker='o')
ax2.set_ylabel("Average Transaction Amount", color='blue')

plt.title("Top 10 Merchant Categories: Fraud Count vs Avg Transaction Amount")
plt.show()
```

C:\Users\karan\AppData\Local\Temp\ipykernel_12252\2423746519.py:20: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.



```
In [152...] sns.barplot(
    x=category_stats.index,
    y=category_stats['fraud_count'],
    hue=category_stats.index, # assign hue same as x
    palette='Reds_r',
    dodge=False,             # avoid splitting bars
    legend=False,            # remove extra legend
    ax=ax1
)
```

Out[152...] <Axes: xlabel='Merchant Category', ylabel='Number of Fraudulent Transactions'>

```
In [154...] ax1.bar(category_stats.index, category_stats['fraud_count'], color='red')
```

Out[154...] <BarContainer object of 10 artists>

```
In [162...] # Import libraries
import pandas as pd
import matplotlib.pyplot as plt

# Ensure plots render in Jupyter
%matplotlib inline

# Load your dataset
df = pd.read_csv("fraud_data.csv")
df.columns = df.columns.str.strip() # Remove extra spaces in column names

# Convert 'amt' and 'is_fraud' to numeric, coerce invalids to NaN
```

```

df['amt'] = pd.to_numeric(df['amt'], errors='coerce')
df['is_fraud'] = pd.to_numeric(df['is_fraud'], errors='coerce')

# Drop rows with missing 'amt' or 'is_fraud'
df_clean = df.dropna(subset=['amt', 'is_fraud']).copy()

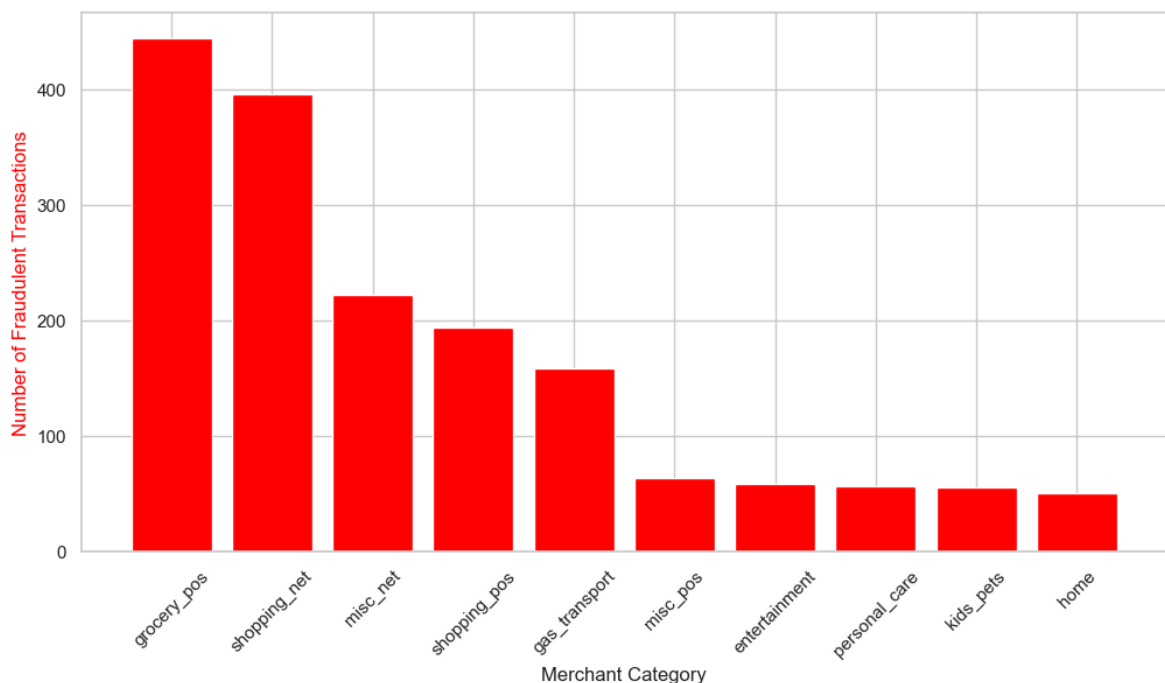
# Aggregate: count of frauds and average transaction amount per merchant category
category_stats = df_clean.groupby('category', observed=True).agg(
    fraud_count=('is_fraud', 'sum'),
    avg_amt=('amt', 'mean')
).sort_values('fraud_count', ascending=False).head(10)

# Check if data is empty
if category_stats.empty:
    print("No data available for plotting!")
else:
    # Create figure and axis
    fig, ax1 = plt.subplots(figsize=(12, 6))

    # Bar chart: fraud count
    ax1.bar(category_stats.index, category_stats['fraud_count'], color='red')
    ax1.set_ylabel("Number of Fraudulent Transactions", color='red', fontsize=12)
    ax1.set_xlabel("Merchant Category", fontsize=12)
    ax1.tick_params(axis='x', rotation=45)

```

In [164... `plt.show()`



In []: